Spécification et preuve de programmes

Master informatique — Année 1

Alain Giorgetti
alain.giorgetti@umlp.fr

Institut FEMTO-ST
Département d'Informatique des Systèmes Complexes

Introduction

- Conception et développement de logiciels sûrs
  - Enjeu économique majeur
  - Fondé sur les méthodes formelles de vérification et validation
- Progression en trois étapes
  - Licence 2 : logique, notations, notion de système formel
  - Master 1 : test, preuve de programmes impératifs (logique de Hoare, annotations, prouveurs)
  - Master 2 : test avancé, model-checking, logiques temporelles, ...

"[...] program verification tools have not reached the same kind of maturity as [...] compilers. It took many years of developing and refining the theory underlying modern compilers, including context-free grammars and data-flow analyses, but these are now taught in undergraduate computer science curricula. We can only hope that program verifiers will eventually become as well understood." [Rustan Leino, préface du KeY Book]

Organisation pratique

- 12 cours, 12 TD, 12 TP de 1 h 30
  - Cours indispensables, pour comprendre les notions et les supports de cours (hétérogènes, non dédiés, *partly in English*)
  - Écoute active : Prenez des notes, posez des questions !
- Travail personnel hebdomadaire : relire les notes de cours et des supports, rédiger des questions, préparer les exercices
- Évaluations
  - Premier partiel début mars (voir ADE), sans documents
  - Second partiel en avril (voir ADE), sur tout le programme
  - TPs notés en temps limité (voir ADE)
- Intervenants : Alain Giorgetti et Nikolai Kosmatov

Programme

Thèmes des cours

1. Vérification déductive
2. Logique de Hoare
3. Annotations JML : conception par contrats pour Java
4. Le prouveur KeY et la logique du premier ordre
5. Java et la logique dynamique
6. Preuves de programmes C spécifiés en ACSL, avec Frama-C, par N. Kosmatov
7. Spécification et vérification de propriétés de haut niveau avec MetAcsl, par N. Kosmatov
8. Preuve de programmes Java avec boucles et tableaux
9. Génération d'obligations de preuves, de Java/JML à JavaDL

3 séances de "marge" pour questions, réponses, exercices, méthodes et approfondissements

+ 6 × 1.5 hour of tutorials in English for SSE and voluntary students (see ADE)

Travaux pratiques

Vérification de programmes Java annotés en JML ou C annotés en ACSL

- Spécification et preuve (automatique, si possible) de fonctions C ou méthodes Java manipulant des entiers et des tableaux
- 5 séances de TP de 90 minutes sur C/ACSL, avec Frama-C, par Nikolai Kosmatov
- 7 séances de TP de 90 minutes sur Java/JML, avec le prouveur KeY
  - Logique du premier ordre avec les types de Java
  - Calcul des séquents
  - Logique dynamique (JavaDL)
  - Preuves de programmes impératifs Java annotés en JML

Références

[Flo67] R. W. Floyd.
Assigning meanings to programs.
In *Symposia in Applied Mathematics*, volume 19, pages 19–32, 1967.

[Hoa69] C. A. R. Hoare.
An axiomatic basis for computer programming.
Commun. ACM, 12(10):576–580, 1969.

[Jul10] J. Julliand.
Cours et exercices corrigés d'algorithmique, vérifier, tester et concevoir des programmes en les modélisant.
Vuibert, 272 pages, 2010.

Cours 1

Vérification déductive

Logique de Hoare

Annotations JML : conception par contrats pour Java

Logique du premier ordre

Java et la logique dynamique

Vérification de programmes itératifs - invariants et variants de boucles

C/ACSL et Frama-C (2 cours)

De Java/JML à JavaDL

Vérification incrémentale

Preuve interactive

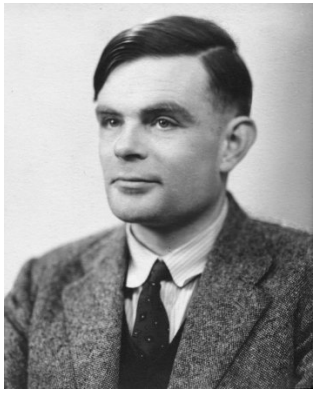
Vérification déductive

Preuve de programmes

Qu'est-ce que la preuve de programmes ?

La *preuve de programmes* est une méthode qui utilise des fondements mathématiques et logiques pour établir rigoureusement des propriétés de programmes.

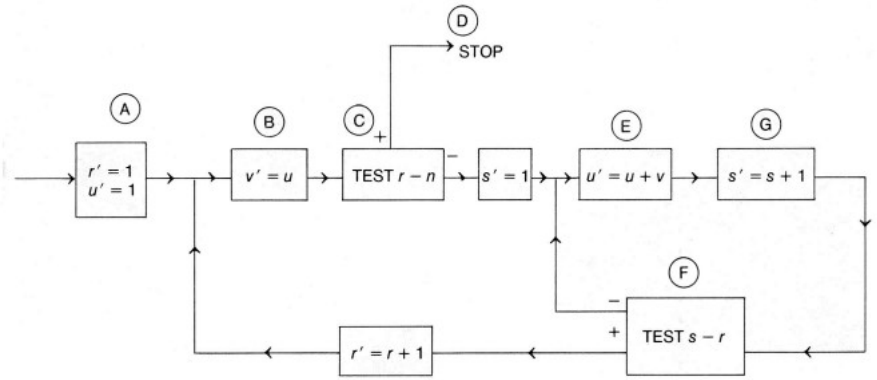
Premiers théoriciens
Alan Turing



Informations

- *Checking a large routine*, 1949
- Calcul de $n!$ sans multiplications
- Par additions successives
- Deux boucles imbriquées
- Preuve de correction
- Notion d'assertion

Routine d'A. Turing

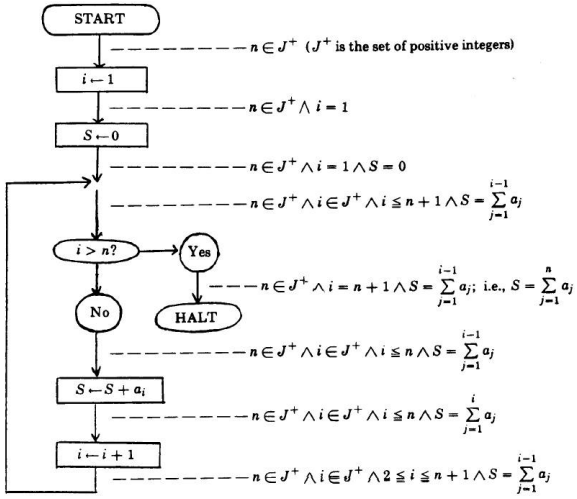


Premiers théoriciens
Robert W. Floyd

Informations

- Théoricien des graphes
- Chercheur en informatique
- Pionnier de la vérification de programme
- Article de 1967: *Assigning Meanings to Programs*

Flowchart



Notions introduites par Robert W. Floyd

Condition de vérification

$$V_c(P; Q)$$

où P et Q sont des annotations et c une étape.

Permet la vérification de la correction d'une étape.

Strongest postcondition

La plus forte postcondition $T_c(P)$ est une postcondition correcte pour la commande c avec la précondition P et elle implique toute postcondition correcte Q.

$$T_c(P) \Rightarrow Q$$

où P et Q sont des annotations et c une étape.
Permet de réduire le nombre d'annotations.
La plus forte postcondition est l'assertion la plus précise possible après l'étape c, lorsqu'elle est précédée par l'assertion P.

Tony Hoare

Informations

- Professeur du Oxford University Computing Laboratory
- A inventé le quicksort
- Logique de Hoare
- Triplet de Hoare
- Article de 1971: *An Axiomatic Basis for Computer Programming*

Logique de Hoare

Triplet de Hoare

Triplet de Hoare

$$P\{Q\}R$$

avec P et R des prédicats et Q une commande

Notation plus moderne :

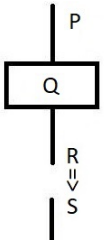
$$\{P\}\ c\ \{Q\}$$

avec P et Q des prédicats et c une commande.

Le triplet de Hoare $\{P\}\ c\ \{Q\}$ est valide si et seulement si, lorsque la précondition P est satisfaite, la commande c établit la postcondition Q, si cette commande termine.

Exemples de règles de déduction

Règle de conséquence



Règles de la postcondition et de la précondition

$$\frac{P\{Q\}R, R \Rightarrow S}{P\{Q\}S}\ postcondition$$

$$\frac{S \Rightarrow P, P\{Q\}R}{S\{Q\}R}\ precondition$$

Où P, R et S sont des prédicats et Q est une instruction.
Permet de passer d'un triplet à l'autre.

Edsger Dijkstra

Informations

- Mathématicien et informaticien
- Algorithme de Dijkstra
- *Weakest precondition*
- Article de 1975: *Guarded commands, nondeterminacy, and formal derivation of programs*

Langages avec des annotations

Floyd, Hoare et Dijkstra utilisaient un "langage de programmation universel".
Dans les années 1970 le premier vrai langage d'annotation est développé: Euclid.
Il sera suivi du langage Eiffel en 1986.
Preuve assistée par des outils comme AutoProof.

Premiers outils

Euclid

```
function Substr(s: String(parameter), first: StringIndex, len: StringLength)
  returns r: String(len) =
      pre (first+len <= s.length+1)
      post {(i in 1..len -> r(i) = s(i+first-1)) and r.length = len}
      begin
      r.length := len
      for i in 1..len loop r(i) := s(i+first-1) end loop
      end Substr

function Catenate(s1: String(parameter), s2: String(parameter), size: StringLength)
  returns r: String(size) =
      pre ((s1.length + s2.length <= size) and
        (r.length = s1.length + s2.length))
      post ((i in 1..s1.length -> r(i) = s1(i)) and
        (i in 1..s2.length -> r(i+s1.length) = s2(i)))}
      begin
      for i in 1 .. s1.length loop r(i) := s1(i) end loop
      for i in 1 .. s2.length loop r(i+s1.length) := s2(i) end loop
      end Catenate
```

Informations

- Langage impératif
- Influencé par le langage Pascal
- Développé par Richard Holt et James Cordy
- Apparu en 1970
- Langage avec des annotations

Premiers outils

Eiffel

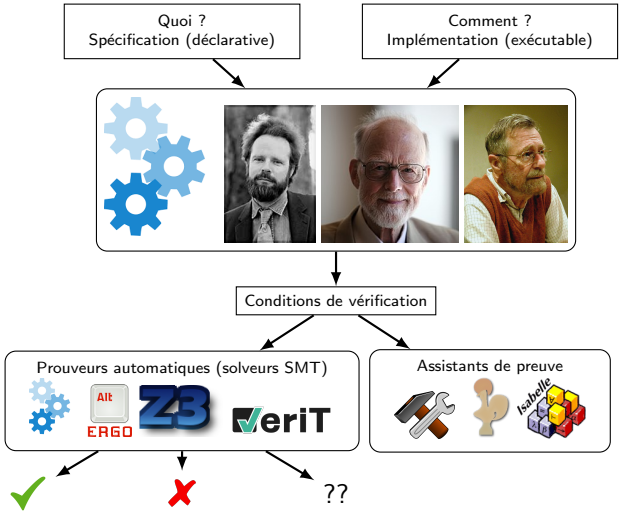
```
pop
    -- Remove top element.
  require
    not empty
  ensure
    not full;
    nb_elements := old nb_elements - 1

change_top (x: T)
    -- Replace top element by x
  require
    not empty
  ensure
    not empty;
    top = x
    nb_elements = old nb_elements
```

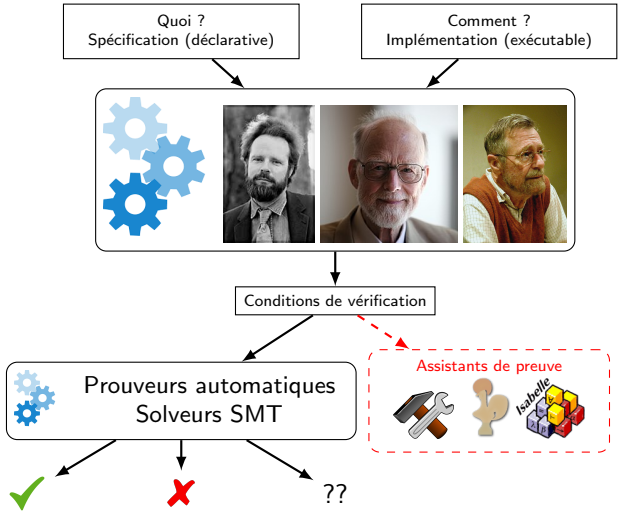
Informations

- Langage de programmation orienté objet
- Développé par Bertrand Meyer
- Apparu en 1986
- A influencé C#, Java et Ruby
- Langage avec des annotations
- Programmation par contrats

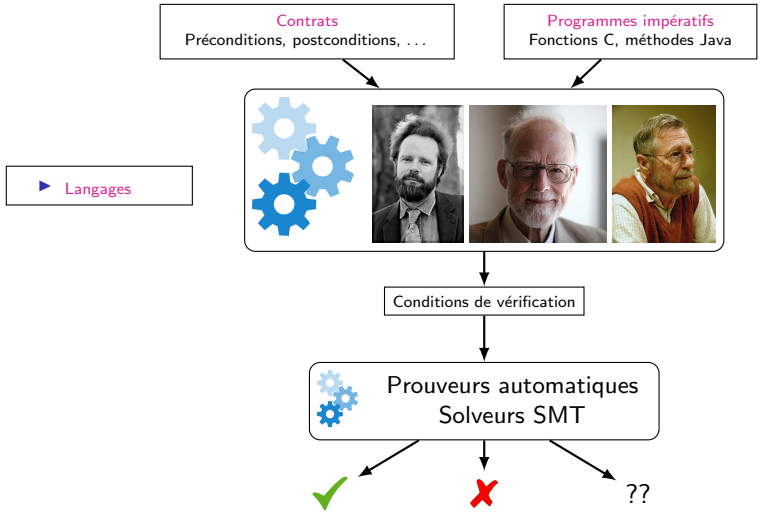
Vérification déductive



Vérification déductive automatique



Programmation impérative, spécification fonctionnelle



Exemple : fonction C, spécification ACSL

Initialisation d'un tableau a de n entiers (fonction identité)

```
/*@ requires n >= 0 && \valid(a+(0..n-1));
    ensures (\forall integer j; 0 <= j < n ==> a[j] == j); */
void id(int a[], int n) {
 /*@ loop invariant 0 <= i <= n;
     loop invariant (\forall integer k; 0 <= k < i ==> a[k] == k);
     loop assigns i, a[0..n-1]; */
 for (int i = 0; i < n; i++) a[i] = i;
}
```

- Fonction C
- Contrat ACSL
- Annotations d'aide à la preuve automatique
- Exemple de condition de vérification (préservation du 2e invariant)

$$(i < n \ \wedge \ \forall k.\ 0 \leq k < i \Rightarrow a[k] = k)$$
$$\Rightarrow (\forall k'.\ 0 \leq k' < i+1 \Rightarrow \mathrm{wr}(a, i, i)[k'] = k')$$

Théorie des tableaux

Comment démontrer

$$(i < n \ \wedge \ \forall k.\ 0 \leq k < i \Rightarrow a[k] = k)$$
$$\Rightarrow (\forall k'.\ 0 \leq k' < i+1 \Rightarrow \mathrm{wr}(a, i, i)[k'] = k')$$

automatiquement ?

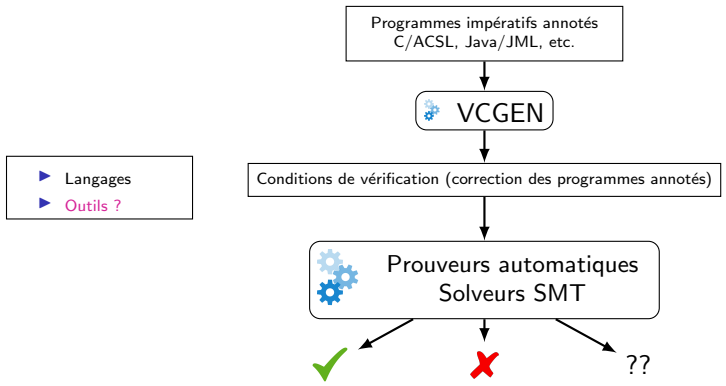
Raisonner dans une théorie du premier ordre, avec SORTES et égalité

- rd : ARRAY × INDEX → ELEM rd(a, i) est noté $a[i]$
- wr : ARRAY × INDEX × ELEM → ARRAY
- Axiomes

$$\begin{array}{rcl} & & \mathrm{wr}(A, I, E)[I] = E \\ I \neq J & \Rightarrow & \mathrm{wr}(A, I, E)[J] = A[J] \end{array}$$

+ théories sur INDEX et ELEM

Programmes impératifs annotés

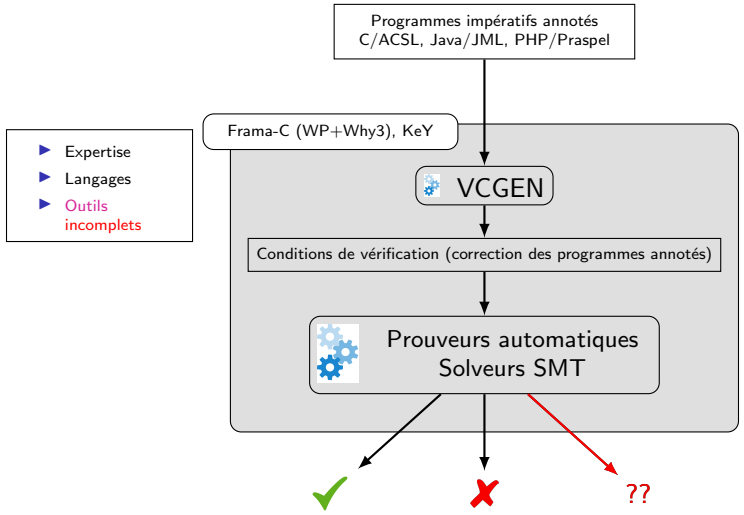


- Langages
- Outils ?

Outils

- Frama-C
  - *Framework for modular analysis of C code*
  - CEA LIST, Inria Saclay
  - C/ACSL (*Ansi C Specification Language*)
  - VCGEN : greffon WP (*Weakest Precondition*)
  - Génération de conditions de vérification avec Why3
  - Appels de solveurs SMT (Alt-Ergo, CVC3, CVC4)
- KeY prover
  - `https://www.key-project.org/`
  - Java/JML (*Java Modeling Language*)
  - Depuis 1998
  - Logique dynamique (JavaDL) adaptée à Java
  - Nombreuses règles de déduction
  - Peut appeler un solveur SMT

"Prouveurs de programmes"



- Expertise
- Langages
- Outils incomplets

Bases

- Algèbre de Boole
- Logique des propositions : $\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$
- Logique du premier ordre (= calcul des prédicats) : $\forall$, $\exists$
- Notions de système déductif et de démonstration formelle

Système déductif, preuve formelle

Rappels, vu en licence (?)

- Un *système déductif* (ou *système formel*) est un ensemble de *règles (de déduction)* de la forme
$$\frac{P_1, \ldots, P_n}{C}$$
- Sur une ligne, la règle s'écrit $P_1, \ldots, P_n \vdash C$
- La règle se lit "C se déduit de P_1 et ... et P_n" ou "Si P_1 et ... et P_n, alors C"
- P_1, ..., P_n sont les *prémisses* de la règle, C est sa *conclusion*
- Le symbole de la déduction $\vdash$ sert aussi à énoncer des théorèmes
- $H_1, \ldots, H_n \vdash C$ veut dire que ..... [R_1]
- Une *démonstration/preuve (formelle)* d'un théorème est un arbre dont
  - la racine est la conclusion du théorème,
  - chaque nœud interne (non feuille) est une déduction qui instancie une règle du système déductif, et
  - chaque feuille est une hypothèse du théorème.
- Un *axiome* est une règle de déduction sans prémisse, donc qui ne produit pas de feuille dans un arbre de preuve

Exemple de système déductif et de démonstration formelle

Dans le système déductif LP dont les règles sont

$$\frac{}{P \Rightarrow (Q \Rightarrow P)}\,(Ax.\ 1) \qquad \frac{}{(P \Rightarrow Q) \Rightarrow ((P \Rightarrow (Q \Rightarrow R)) \Rightarrow (P \Rightarrow R))}\,(Ax.\ 2)$$

...

$$\frac{}{\neg\neg P \Rightarrow P}\,(Ax.\ 9) \qquad \frac{}{(P \Rightarrow Q) \Rightarrow ((P \Rightarrow \neg Q) \Rightarrow \neg P)}\,(Ax.\ 10) \qquad \frac{P, P \Rightarrow Q}{Q}\,(m.p.)$$

une démonstration formelle du théorème $A, \neg A \vdash B$ est :

$$\dfrac{\dfrac{\dfrac{\neg A \qquad \dfrac{}{\neg A \Rightarrow (\neg B \Rightarrow \neg A)}\,Ax.\ 1}{\neg B \Rightarrow \neg A}\,m.p. \qquad (1)}{\neg\neg B}\,m.p. \qquad \dfrac{}{\neg\neg B \Rightarrow B}\,Ax.\ 9}{B}\,m.p.$$

où (1) est le sous-arbre

$$\dfrac{\dfrac{A \qquad \dfrac{}{A \Rightarrow (\neg B \Rightarrow A)}\,Ax.\ 1}{\neg B \Rightarrow A}\,m.p. \qquad \dfrac{}{(\neg B \Rightarrow A) \Rightarrow ((\neg B \Rightarrow \neg A) \Rightarrow \neg\neg B)}\,Ax.\ 10}{(\neg B \Rightarrow \neg A) \Rightarrow \neg\neg B}\,m.p.$$

Présentation dans un tableau

No	Formule	Justification
1	$\neg A$	Hypothèse 2
2	$\neg A \Rightarrow (\neg B \Rightarrow \neg A)$	Axiome 1
3	$\neg B \Rightarrow \neg A$	m.p. 1, 2
4	A	Hypothèse 1
5	$A \Rightarrow (\neg B \Rightarrow A)$	Axiome 1
6	$\neg B \Rightarrow A$	m.p. 4, 5
7	$(\neg B \Rightarrow A) \Rightarrow ((\neg B \Rightarrow \neg A) \Rightarrow \neg\neg B)$	Axiome 10
8	$(\neg B \Rightarrow \neg A) \Rightarrow \neg\neg B$	m.p. 6, 7
9	$\neg\neg B$	m.p. 3, 8
10	$\neg\neg B \Rightarrow B$	Axiome 9
11	B	m.p. 9, 10